You can apply `DataAnnotations` attributes directly to your parameter types to indicate the type of data that's acceptable. This approach allows you to check that required fields have been provided, that numbers are in the correct range, and that email fields are valid email addresses, for example.

Consider the checkout page for a currency-converter application. You need to collect details about the user—their name, email, and (optionally) phone number—so you create an API to capture these details. The following listing shows the outline of that API, which takes a `UserModel` parameter. The `UserModel` type is decorated with validation attributes that represent the validation rules for the model.

Listing 7.10 Adding `DataAnnotations` to a type to provide metadata

```
using System.ComponentModel.DataAnnotations;                    ❶

WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

app.MapPost("/users", (UserModel user) => user.ToString());     ❷

app.Run();

public record UserModel
{
    [Required]                                                  ❸
    [StringLength(100)]                                         ❹
    [Display(Name = "Your name")]                               ❺
    public string FirstName { get; set; }

    [Required]
    [StringLength(100)]
    [Display(Name = "Last name")]
    public string LastName { get; set; }

    [Required]
    [EmailAddress]                                              ❻
    public string Email { get; set; }

    [Phone]                                                     ❼
    [Display(Name = "Phone number")]
```